既有專案與模型組合：Coding Agent Harness 的第二次實證評估

Coding_agent_harness 技術報告（二）｜2026 年 10 月 10 日｜評測對象：harness v4.1（agent 程式的雜湊 5b37b79521737866）、v4.1.1（ed861606750f20b1）；評測後的改版 v4.2（6436be794225c705）｜模型：gemma4:26b、Qwen2.5-Coder-32B-Instruct、DeepSeek-R1-Distill-Qwen-32B（本機 Ollama，Q4_K_M）

摘要　第一次研究用 DevBench（只給規格、從零寫出整個專案）評估這個 coding harness；這次補上日常更常見的情況——在既有的真實專案裡照 issue 修正——並測試一般認為 coding 很強的模型組合。既有專案用 SWE-bench 的 24 題子集（Python 16、C 4、C++ 4），在官方 Docker 環境裡執行、用官方套件計分；從零寫沿用 DevBench 的 12 個 repo。條件包括 gemma4:26b、DeepSeek-R1-32B 規劃＋Qwen2.5-Coder-32B worker、Qwen 自己規劃，以及同一個模型不用 harness 的對照組。結果：(1) 在大型真實專案上，harness 解決 7～8/24（29～33%），直接寫 6/24（25%），沒有可信的差別；harness 比較能找到該改的檔案（18/24 對 12/24），但改對的沒有比較多，自己的驗證也跟官方結果幾乎無關。(2) R1＋Qwen 在 DevBench 明顯比 gemma4:26b 差（單元測試 3.0% 對 14.7%，p=0.016），在 SWE-bench 差不多（8/24 對 7/24）；主因是 harness 的寫檔協議跟 Qwen 的習慣不合（11～16% 的模型呼叫在空轉），R1 當規劃者也沒有幫助。(3) deepseek-r1:32b 的 thinking 關不掉；gemma4:26b 在簡單的照格式問答上開 thinking 沒有變準。(4) 評測中找到 C++ 掃描器的災難性回溯（已修正）與大型檔案的修改視窗問題。據此做的 v4.2 讓 SEARCH 對不上少了 76%、每題時間少了一半以上，但解決數不變（7/24），DevBench 也沒有可信的變化；它也暴露出新的問題：驗證沒有跑到任何測試，卻判定通過。文獻與本研究的數據都指向同一個做法：規劃階段的 self-refine 應該是「用程式檢查計畫、把具體的問題交回規劃者」，而不是請模型自己再想一次。

1. 引言

1.1 背景與問題

第一次研究（`benchmark_study/`）的結論是：這個 harness 可以擔任「有測試的日常修改」的助手，還不能擔任「只給規格的長任務」——在公開基準 DevBench 上，gemma4:26b 的單元測試通過率只有 10～24%。這留下兩個問題：

1. **DevBench 量的是「從規格從零寫出整個專案」**，連 GPT-4-Turbo 都只有個位數的通過率 [1]。日常的開發工作多半是另一種：在既有的專案裡，照一個問題描述修改。在既有的真實專案裡，harness 的表現會不會不同？
2. **換成一般認為 coding 很強的模型會不會更好？** 一個常見的建議（使用者與 Gemini 討論的結論）是：規劃者用會思考的推理模型 DeepSeek-R1，worker 用程式專用的 Qwen2.5-Coder-32B。

另外兩個工程問題：規劃階段要不要也加上 self-refine（像 worker 用測試精煉結果一樣）？Web Console 能不能分別設定每個角色的模型與 thinking 模式？

1.2 研究問題

- **RQ1（既有專案）**：在真實既有專案的 issue 修正上（SWE-bench 子集 24 題，Python／C／C++），harness 能解決多少？比直接用同一個模型好多少？
- **RQ2（模型組合）**：DeepSeek-R1 規劃＋Qwen2.5-Coder worker，在從零寫（DevBench）與既有專案（SWE-bench）上，比 gemma4:26b 好嗎？
- **RQ3（推理模型當規劃者）**：規劃者換成會思考的 R1，比 Qwen 自己規劃好嗎（worker 都是 Qwen）？
- **RQ4（harness 對程式專用模型的價值）**：Qwen2.5-Coder 用 harness 比直接寫好多少？
- **RQ5（thinking 與規劃的 self-refine）**：各角色該不該開 thinking？規劃階段的 self-refine 值不值得加進框架？
- **RQ6（改版）**：根據這次找到的問題修改之後（v4.2），既有專案與從零寫的結果有沒有變好？

1.3 本報告的貢獻

1. 一套在單機上用**官方 Docker 映像**評測既有專案的方法，harness 本身不用修改（4.2 節）；第一次研究因為需要 Docker 而沒有採用 SWE-bench。
2. 三種模型組合在兩類任務上的比較，每一組都有「不用 harness」的對照組，並用配對比較與信賴區間分析。
3. 從 job 的事件流算出的 **worker 行為指標**（工具錯誤率、卡住、空轉的修改、修改對不上的位置），用來解釋「為什麼換模型沒有變好」「為什麼在大型專案上沒有贏」，而不是只看分數。
4. 評測中找到的兩個 harness 問題（C++ 掃描器的災難性回溯、大型檔案的修改視窗），以及根據證據做的改版 v4.2 與它的量測。
5. 對 Gemini 建議的逐項檢查、thinking 模式的實測，與規劃階段 self-refine 的評估。

2. 背景與相關研究

2.1 兩類基準：從零寫，還是在既有專案裡改

DevBench [1] 屬於「從規格從零寫出整個專案」：agent 拿到需求文件、UML 與架構設計，要寫出全部的原始碼，用官方的單元與驗收測試計分。這類任務對所有模型都難——DevEval 公開的 GPT-4-Turbo 驗收 7.1%、單元 8.0% [1]，NL2Repo-Bench 報告最強的 agent 平均測試通過率也低於 40% [2]。

日常的工作多半是**在既有的專案裡，照一個問題描述修改**，最常用的基準是 SWE-bench [3]：每一題是 GitHub 上真實的 issue 與修正它的 PR；agent 看得到 issue 與修正前的整個 repo，評分時套上 agent 的 patch，跑 PR 附帶的測試——`FAIL_TO_PASS`（修正前失敗、修正後要通過）與 `PASS_TO_PASS`（原本通過、不能被弄壞），兩種全部通過才算解決（resolved）。SWE-bench Verified [4] 是其中 500 題經過人工確認題目描述清楚、評分測試合理的子集；SWE-bench Multilingual [5] 用同樣的方法收集了 9 種語言、42 個 repo 的 300 題（包括 C 與 C++）。

這類任務的難處跟從零寫不一樣：規格就是一段 issue（常常只描述現象），repo 很大（django 有將近 5,000 個檔案），要先**找到該改的地方**，再改對、而且不弄壞別的東西。公開成績的量級（pass@1，SWE-bench Verified）：GPT-4o 配 Agentless [19] 38.8%、配 SWE-agent [20] 23.0%；Claude 3.7 Sonnet 配 SWE-agent 58.2% [5]。**Qwen2.5-Coder-32B-Instruct 沒有微調、配 OpenHands [21] 只有 7.0%，而且 29.4% 的題目卡在重複動作的迴圈裡** [6]；用 agent 軌跡微調後才到 20.6% [6]～40.2% [5]。SWE-bench Multilingual 上，Claude 3.7 Sonnet 43%、Qwen2.5-Coder-32B-Instruct 6.5% [5]。也就是說，開源的 32B 模型在這類任務上本來就很低，主要的提升方法是用 agent 軌跡微調。

2.2 這次換的兩個模型

- **Qwen2.5-Coder-32B-Instruct** [7]：程式專用的 32B 模型，原生 context 32K。報告的成績：HumanEval 92.7、Aider 程式編輯 pass@1 60.9（同一張表 GPT-4o 56.8）[7]。沒有 thinking 模式。
- **DeepSeek-R1** [8]：用強化學習訓練出長鏈推理的模型；原版（671B）SWE-bench Verified 49.2%、Aider-Polyglot 53.3 [8]。這次用的 `deepseek-r1:32b` 是把 R1 的推理蒸餾到 Qwen2.5-32B 的版本，論文只量了它的數學與程式競賽（AIME 72.6、LiveCodeBench 57.2），**沒有任何軟體工程的評測** [8]。R1 的報告在限制一節寫明：結構化輸出（JSON）的能力不如既有模型、不會使用工具、軟體工程任務相對 DeepSeek-V3 沒有明顯進步、few-shot 提示會讓表現變差 [8]。

2.3 規劃與自我修正

讓模型「產生 → 自己批評 → 修改」的方法很多：Self-Refine [9]、Reflexion [10]、Self-Debug [11]。但後續研究對「沒有外部回饋的自我修正」大多是負面的：Huang 等人 [12] 發現在推理任務上，沒有外部回饋時模型很難改對、有時反而變差；Kamoi 等人的綜述 [13] 整理出——除了特別適合的任務，沒有研究證明「用提示讓模型自己給回饋」能成功自我修正，有可靠的外部回饋時才有效。在程式上，Olausson 等人 [14] 發現把修正的成本算進去之後，自我修復的收益常常不大，瓶頸是模型給自己的回饋品質。針對規劃，Kambhampati 等人 [15] 主張模型自己不能可靠地規劃或驗證計畫，應該讓模型產生候選、由外部的驗證器檢查並回饋（LLM-Modulo）。在 repo 層級的修改上，CodePlan [16] 把修改視為規劃問題、用依賴分析決定下一步；Self-planning [17] 與 PlanSearch [18] 先產生自然語言的計畫再寫程式。

3. 系統與這次的修改

評測的對象是 harness v4.1——第一次研究最後的版本；agent 程式（`agent_core/`、`skills_system/`、`coding_agents/`、`context_modes/`、`web_app/`、`AGENT.md`）的雜湊 5b37b79521737866 跟第一次研究的 v4.1 快照完全相同。評測都從凍結的副本執行（`artifacts/` 有快照），評測期間在主目錄做的修改不影響結果。

coding orchestrator 的流程跟第一次研究相同：探索專案（`repo_map`）→ 規劃 session 挑檔案讀 → 規劃 session 拆成 work items，由程式檢查與修補計畫 → 核准（評測時自動）→ 每一項派一個 worker：**決策 session** 以 JSON 決定下一個動作，要寫檔案時由另一個 **內容通道** session 產生檔案內容 → 每一項由程式驗證（語法、未定義名稱、跨檔匯入、這一項的驗證指令）→ 失敗交還 worker、再換新的 worker 重試 → 整體驗證（失敗就派修正、變差就還原）→ 報告。

這次對 harness 的修改（都在評測之外完成）：

1. **每個角色各自的模型**（預設行為不變）：Web Console 標題列的「🧠 模型」可以分別設定主對話（非 coding）、Coding 規劃、Coding worker 的模型（也可以用 `/model`、`/code_model`）。主對話的設定存在 `logs/model_settings.json`，啟動時載入、比環境變數 `WEB_CONSOLE_MODEL` 優先；摘要模型沒有另外指定時跟著主對話的模型換。
2. **每個角色各自的 thinking 開關**（預設全部關閉）：只在模型支援 thinking 時顯示（依 Ollama 回報的模型能力）；送出前一律檢查，模型不支援就送 false（不支援的模型收到 true 會回 400）。主對話的思考內容顯示在右欄、不進對話；worker 開 thinking 時內容通道的輸出上限多給 8,192 tokens。測試：主對話的自我測試新增 2 項、瀏覽器自動化測試新增 2 項，全部通過（主對話 12/12、Web UI 8/8）；也用真的模型（gemma4:e4b）確認主對話開 thinking 時回覆仍是合法的 JSON、思考內容有被取出。
3. **v4.1.1**：修正評測中找到的 C++ 掃描器的災難性回溯（5.1 節），一行。
4. **v4.2**：根據既有專案的結果修改四件事（5.5 節）。
5. **既有專案的評測工具** `evals/swebench/`（4.2 節）。

4. 方法

4.1 資料

- **DevBench 子集**（E8）：跟第一次研究同樣的 12 個 repo（Python 7、C++ 5），官方測試 agent 看不到，計分方式不變（單元測試 116 個、驗收測試 51 個）。
- **SWE-bench 子集**（E9）：24 題——Python 16 題來自 Verified，C 4 題（redis 2、jq、micropython）與 C++ 4 題（fmt 3、nlohmann/json）來自 Multilingual。抽樣規則事先寫好、不看題目內容：Python 依 Verified 的專案分布與人工標註的難度分布分層抽樣（修正大約 15 分鐘內 6 題、15 分鐘～1 小時 8 題、1 小時以上 2 題；django 6 題、sympy 與 sphinx 各 2 題、其他專案各 1 題），C／C++ 依專案配額隨機抽，固定種子 20261009（附錄 A；`datasets/swebench_subset/select_tasks.py`）。

4.2 既有專案的執行環境

每一題用官方的 Docker 映像當「已經裝好的開發環境」，harness 本身不用修改（`evals/swebench/env.py`）：

1. 從映像起一個容器（`--network none`，不能連網路），主機上的工作區掛在容器裡同一個路徑；映像裡的 repo（`/testbed`）複製到工作區，editable install 的路徑也改到工作區——容器裡 import、建置、跑測試用的就是工作區的檔案。
2. `python`、`pytest`、`make`、`cmake`、`gcc`、`tclsh` 等同名的包裝指令用 `docker exec` 在容器裡、同一個工作目錄執行。harness 的 `code_test`／`code_run`／`code_check` 會把 `CODE_PYTHON` 所在的目錄放在 PATH 最前面，所以設 `CODE_PYTHON` 指向包裝目錄，harness 跑的測試、建置與編譯檢查就都在題目的環境裡；讀寫檔案照常在主機上。
3. 工作區根目錄放一份開發環境說明 `.coding_agent.md`（harness 原本就會讀的專案規範檔，相當於開發者寫的 AGENTS.md），只寫「怎麼建置、怎麼跑測試」的通用格式（例如 django 用 `python tests/runtests.py <測試模組>`），例子刻意避開評分用的測試。
4. 任務文字是 issue 原文加上固定的要求；跟 SWE-bench 的標準設定一樣不給 `hints_text`。agent 看不到評分用的測試與參考解答。
5. 結束時，工作區相對於 `base_commit` 的改動（agent 新寫的原始檔也算；建置產物、二進位檔與 `.coding_agent.md` 不算）就是 patch，交給官方的 swebench 套件（5.0.2）在新的容器裡套用、執行評分腳本、判斷 resolved。

冒煙測試（jq 一題）確認了整條流程：harness 在容器裡建置與跑 `make check`、patch 由官方計分套用並跑評分測試。計分環境用參考解答驗證（5.1 節）。

4.3 條件

條件	規劃	worker	說明
harness：gemma4:26b	gemma4:26b	gemma4:26b	第一次研究的建議設定（DevBench 另有第一次研究的兩次結果）
harness：R1 規劃＋Qwen worker	deepseek-r1:32b（thinking 開）	qwen2.5-coder:32b	使用者想測的組合
harness：Qwen	qwen2.5-coder:32b	qwen2.5-coder:32b	跟上一列只差規劃者：量 R1 當規劃者的效果（RQ3），只跑 DevBench
不用 harness：Qwen／gemma4:26b	—	同一個模型	DevBench：一個檔案一次呼叫直接寫（第一次研究 E2 的作法）；SWE-bench：BM25 檢索相關的原始檔＋一次寫出修改（SWE-bench 論文的 RAG 基準作法）

SWE-bench 的「不用 harness」：以 issue 文字為查詢，對 repo 的原始檔（不含測試、文件、範例、第三方程式）算 BM25，依分數放進提示，總長度上限 6 萬字元（約 1.8 萬 tokens；太長的檔案只放跟 issue 最相關的兩段），模型一次輸出 SEARCH/REPLACE 修改，用跟 harness 的 `code_edit` 一樣的比對方式套用。這是一個不弱的對照組：在先試的 8 題裡，有 7 題檢索到了參考解答要改的檔案。

4.4 硬體與模型設定

RTX 5090（32 GB）、123 GB 記憶體。評測期間 Isaac Sim 同時在跑（佔 6.2 GB 顯示記憶體、約 40% GPU 使用率）。四個模型都是 Ollama 預設的 Q4_K_M 量化（4-bit；公開成績多半是全精度）。32B 模型在預設的 f16 KV cache、32K context 下放不進剩下的顯示記憶體：4 GB 要放到 CPU，只有 18 tokens/秒；所以另外開一個評測專用的 Ollama（同一份模型檔）設 `OLLAMA_KV_CACHE_TYPE=q8_0`，完整放進 GPU，約 42 tokens/秒。第二次研究的全部條件都跑在這個設定；第一次研究的 gemma4:26b 結果跑在預設的 f16。兩個 32B 模型放不下同一張卡，R1 規劃與 Qwen worker 交替時 Ollama 要換模型（每次約 10～20 秒，計入時間）。harness 的設定跟第一次研究相同（num_ctx 32768、temperature 0.2）。時間上限：DevBench 每個 repo 2 小時、SWE-bench 每一題 1 小時。

4.5 指標與統計

- DevBench：官方單元與驗收測試的合併通過率；以 repo 為單位重抽樣的 bootstrap 95% 區間（B=10,000）；同一個條件跑好幾次時，每個 repo 把幾次的通過數與總數加總。兩個條件同一個 repo 配對比較：差值的配對 bootstrap 區間、勝負的符號檢定。
- SWE-bench：resolved 的比例與 Wilson 95% 區間；兩個條件同一題配對，用精確 McNemar 檢定（只看一對一錯的題目）。另外記錄 patch 有沒有改到參考解答改的檔案（定位）、harness 的 worker 有沒有看過那個檔案、patch 的大小（原始碼與測試分開算）、harness 自己的整體驗證跟官方結果一不一致。
- worker 行為（從 job 的事件流算）：工具錯誤率、驗證失敗、卡住被結束的 worker、空白回覆、空轉的修改（「新內容跟檔案目前的內容完全一樣」）、換新 worker 重試；模型呼叫與時間依角色（規劃／worker）分開算。

5. 結果

5.1 計分環境的驗證，與評測中找到的兩個 harness 問題

計分環境。24 題的參考解答全部 resolved。其中 fmt-1683 第一次是 P2P 22/42：當時 harness 與 3 個平行計分同時在跑，評分腳本 `set -x` 的追蹤行（stderr）跟測試輸出（stdout）交錯，結束標記比後面 20 個測試的結果更早寫進紀錄，官方的 log parser 只讀標記之間的內容，就把它們算成沒通過；單獨重跑是 42/42。因此每一組結果都用同一條規則檢查「結束標記之後還有測試結果」的題目（`evals/swebench/recheck.py`）；全部 5 組 agent 的結果都沒有可疑的題目。

問題一：C++ 大綱掃描器的災難性回溯（已修正，v4.1.1）。SWE-bench 跑到 C／C++ 題目時，fmt-3901 的 `repo_map` 跑了 7 分多鐘，redis 兩題與 fmt-1683 跑滿 1 小時（redis 兩題的 patch 是空的）。用 faulthandler 定位到

`_cpp_common.parse_signature` 裡判斷函式簽名後綴的正規表示式 `_POST_RE`：巨集名稱的分支 `[A-Z_][A-ZO-9_]*` 放在可以重複、中間空白可以是零個的群組裡，一串大寫的名稱可以被切成任意多段，後面一對不上就把每一種切法都試一遍。fmt 內附的 gmock.h 前 2,000 行 0.02 秒、前 4,000 行 216 秒。用單檔 10 秒的上限掃過全部題目：fmt（gmock 兩個檔）、redis（jemalloc 的 3 個標頭）、nlohmann/json（4 個，含 doctest.h）、astropy（expat.h）共 7 題受影響；jq、micropython、其他 Python 題目、DevBench 的 5 個 C++ repo 都沒有——所以 DevBench 與第一次研究的結果不受影響。影響範圍：`repo_map` 每次卡到腳本的逾時（10 分鐘）；規劃時建符號索引在 orchestrator 自己的程序裡、沒有逾時，可以卡到整個 job 逾時。修正只有一行（分支前後加 `\b`，名稱不能從中間切開），gmock.h（1.1 萬行）0.09 秒，`coding_selftest` 加了回歸測試。

處理方式：gemma4:26b 的 v4.1 執行在第 21 題後停掉，受影響的 7 題用 v4.1.1（只差這一行）重跑；其餘 17 題沒有任何會觸發的檔案，兩版的程式路徑完全相同，直接沿用。R1＋Qwen 的 24 題全部用 v4.1.1。受影響的題目在 v4.1 跑完的 4 題（astropy、redis ×2、fmt-1683）0 題解決、3 題跑滿 1 小時；用 v4.1.1 重跑的 7 題解決 2 題（fmt-2310、nlohmann/json-4237），平均 8.2 分鐘、沒有任何一題逾時。

問題二：大型檔案的修改視窗只有前 800 行（v4.2 修正）。檔案超過 250 行時，worker 用 SEARCH/REPLACE 修改，而產生修改內容的 session 只看得到檔案的**前 800 行**（`coding_worker._content_prompt`）。要改的地方在第 800 行之後時，模型看不到原文，只能憑印象寫 SEARCH，對不上就重來，連續三次同樣的結果就被判卡住。gemma4:26b 的 24 題裡「SEARCH 對不上」發生 125 次：44 次（6 題）最像的段落在第 800 行之後，30 次在檔案裡找不到任何相似的段落；R1＋Qwen 是 103 次、41 次、18 次（表 1）。每一次至少花掉兩次模型呼叫，合計佔模型呼叫的 12% 以上。例：xarray-3677 的計畫完全正確（先寫重現問題的測試，再修 `merge.py`），修正那一項的 SEARCH 連續對不上、被判卡住而跳過，最後的 patch 只有測試；django-11734 要改 2,000 多行的 `sql/query.py`，patch 也只有測試。DevBench 與自建情境的檔案都很短，所以先前從來沒有踩到。

表 1　寫檔的白工（SWE-bench）：空轉與 SEARCH 對不上

條件	job 數	空轉（內容一模一樣）	空轉佔模型呼叫（下限）	SEARCH 對不上	其中目標在第 800 行之後	其中找不到相似的段落	SEARCH 對不上佔模型呼叫（下限）
harness：gemma4:26b	23	31	3.0%	125	44	30	12.1%
harness：DeepSeek-R1-32B 規劃＋Qwen2.5-Coder-32B worker	24	39	4.7%	103	41	18	12.5%
harness：gemma4:26b（v4.1，C++ 掃描器 bug）	1	2	4.1%	2	0	0	4.1%
harness：gemma4:26b（v4.2）	24	18	2.5%	30	7	16	4.1%

整體：harness＋gemma4:26b 解決 7/24（29.2%，95% CI 14.9～49.2），harness＋R1 規劃＋Qwen 8/24（33.3%，18.0～53.3），同一個模型直接寫（BM25 檢索＋一次寫出修改）gemma 6/24、Qwen 6/24（25.0%，12.0～44.9）（表 2）。逐題配對（表 3）：每一對比較都看不出差別（McNemar p 0.73～1.0），而且解決的題目重疊很少——例如 gemma 的 harness 與直接寫，只有 harness 解決 5 題、只有直接寫解決 4 題、兩者都解決 2 題——表示單次執行的結果受隨機性影響很大，這個題數無法分辨 5～10 個百分點的差距。

表 3　SWE-bench 子集：同一題的配對比較

比較（A 對 B）	題數	A／B 解決	只有 A 解決／只有 B 解決	都解決	McNemar p
gemma4:26b：harness 對不用 harness	24	7／6	5／4	2	1.000
R1＋Qwen 的 harness 對 Qwen 直接寫	24	8／6	5／3	3	0.727
R1＋Qwen 對 gemma4:26b（都用 harness）	24	8／7	4／3	4	1.000
不用 harness：Qwen 對 gemma4:26b	24	6／6	2／2	4	1.000
改版：gemma4:26b 的 v4.2 對 v4.1／v4.1.1	24	7／7	4／4	3	1.000
gemma4:26b：v4.2 的 harness 對不用 harness	24	7／6	4／3	3	1.000

定位比較好、改對沒有比較好（表 5）：gemma 的 harness 改到參考解答要改的檔案 18/24（worker 看過那個檔案 19/24），直接寫 12/24（gemma）、14/24（Qwen）；harness 從來不交白卷（直接寫有 6／4 題什麼都沒改）。但是 gemma 的 harness 有 9 題「改對檔案但沒修好」、4 題「修好了（F2P 全過）但弄壞既有測試」，直接寫（gemma）分別是 6 與 1。R1＋Qwen 的 harness 定位較差（改到 14/24、7 題改錯地方），但修好的比較乾淨（只有 1 題弄壞既有測試）。

表 5　SWE-bench 子集：沒有解決的原因

條件	解決	沒有改任何東西	patch 套不上	改錯地方（沒改到該改的檔案）	改對檔案但沒修好	修好了但弄壞既有測試
harness：gemma4:26b	7	0	0	4	9	4
harness：DeepSeek-R1-32B 規劃＋Qwen2.5-Coder-32B worker	8	2	0	7	6	1
不用 harness：gemma4:26b	6	6	0	5	6	1
不用 harness：Qwen2.5-Coder-32B	6	4	0	4	8	2
harness：gemma4:26b（v4.1，C++ 掃描器 bug）	0	2	0	0	1	1
harness：gemma4:26b（v4.2）	7	1	0	4	9	3

harness 自己的驗證在大型專案上幾乎沒有參考價值（表 7）：gemma 的 harness 整體驗證通過的 4 題裡 3 題官方沒有解決，整體驗證沒過的 19 題裡 5 題官方其實解決了；R1＋Qwen 整體驗證通過的 3 題全部沒有解決，沒過的 21 題裡 8 題其實解決了。原因很具體：(1) 測試指令選錯——`repo_map` 依專案結構建議 `python3 -m pytest -q`；django 的 6 題規劃者都照 `.coding_agent.md` 用了 `runtests.py`，sympy 的 2 題卻都選了 pytest（說明寫的是 `python bin/test`，而 sympy 的環境沒有 pytest，驗證永遠失敗）；(2) 整體驗證跑整個專案的測試（django 有 2 題不帶測試模組跑全部、其他專案多半是不帶參數的 `code_test`），太慢或有跟這個 issue 無關的失敗；(3) agent 自己寫的測試照自己的理解寫，自己的錯誤行為也會讓自己的測試通過（django-12276：「3 passed」）。harness 最大的優勢——「驗證、失敗就修」——在這種情況下收到的是錯誤的訊號。

表 7　harness 自己的整體驗證與官方結果

條件	題數	自己驗證通過、官方也解決	自己驗證通過、官方沒解決	自己驗證沒過、官方解決	都沒過	通過的題目中沒有跑到任何測試	「通過」可信的比例	「沒過」可信的比例
harness：gemma4:26b	23	1	3	5	14	0	1/4	14/19
harness：DeepSeek-R1-32B 規劃＋Qwen2.5-Coder-32B worker	24	0	3	8	13	0	0/3	13/21
harness：gemma4:26b（v4.1，C++ 掃描器 bug）	1	0	0	0	1	0	—	1/1
harness：gemma4:26b（v4.2）	24	6	10	1	7	3	6/16	7/8

表 9　時間與模型呼叫的分配（SWE-bench）

條件	題數	規劃呼叫	規劃分鐘	worker 呼叫	worker 分鐘	載入模型秒數	計畫項數	計畫被程式修補
harness：gemma4:26b	23	2.0	0.1	88.0	4.8	0	3.1	2/23
harness：DeepSeek-R1-32B 規劃＋Qwen2.5-Coder-32B worker	24	2.0	0.8	66.9	7.3	7	2.6	3/24
harness：gemma4:26b（v4.1，C++ 掃描器 bug）	1	2.0	0.1	95.0	6.0	0	3.0	0/1
harness：gemma4:26b（v4.2）	24	2.0	0.1	59.1	3.0	0	2.4	1/24

R1＋Qwen 比 gemma4:26b 差（表 10、表 11）：單元測試 3.0%（兩次：4.3%、1.7%）對 14.7%（三次：24.1%、10.3%、9.5%），同一個 repo 配對 −11.6 個百分點（95% CI −23.5～−4.1；12 個 repo 中 0 勝 7 負 5 平，p=0.016）；驗收測試 11.8% 對 23.5%（1 勝 8 負，p=0.039）。這是這次研究中唯一達到顯著的模型比較。

R1 當規劃者沒有幫助：worker 都是 Qwen，R1 規劃 3.0%、Qwen 自己規劃 19.0%（−15.9 個百分點，95% CI −40.3～0.0，0 勝 3 負 9 平，p=0.25）。Python 的 7 個 repo 三次的分數幾乎一樣——已確認兩邊的計畫與寫出來的程式都不同，分數一樣是因為 Qwen worker 的失敗是系統性的（例如 lice 三次的測試都在 import 時就失敗：少了規格裡的 `LICENSES` 或 `valid_year`）；差距主要來自 graph-cpp（Qwen 規劃 16/22、R1 規劃兩次都 0/22），也就是第一次研究就看到的「文件自己寫了 setVexer／setVexes 兩種名稱，選對選錯」。所以合理的結論是「沒有證據顯示 R1 規劃比較好」，而不是「R1 規劃比較差」。

harness 對兩個模型都有幫助（單元測試）：gemma4:26b +10.3 個百分點（95% CI 3.5～20.5，7 勝 0 負，p=0.016）；Qwen +15.5（95% CI 0.0～40.2，3 勝 0 負，p=0.25，只有一次）。驗收測試則沒有差別。不用 harness 時兩個模型差不多（單元 3.4% 對 4.3%）。

表 11　DevBench：同一個 repo 的配對比較

比較（A 對 B）	測試	A−B	95% CI	repo 勝／負／平	符號檢定 p
換成「coding 很強」的模型組合（同樣用 harness）	單元	-11.6 個百分點	[-23.5, -4.1]	0／7／5	0.016
換成「coding 很強」的模型組合（同樣用 harness）	驗收	-11.8 個百分點	[-34.1, 10.9]	1／8／3	0.039
規劃換成 R1（worker 一樣是 Qwen）	單元	-15.9 個百分點	[-40.3, 0.0]	0／3／9	0.250
規劃換成 R1（worker 一樣是 Qwen）	驗收	+0.0 個百分點	[0.0, 0.0]	0／0／12	1.000
Qwen 用 harness 對不用 harness	單元	+15.5 個百分點	[0.0, 40.2]	3／0／9	0.250
Qwen 用 harness 對不用 harness	驗收	-2.0 個百分點	[-7.7, 0.0]	0／1／11	1.000
R1＋Qwen 的 harness 對 Qwen 直接寫	單元	-0.4 個百分點	[-4.5, 2.6]	1／1／10	1.000
R1＋Qwen 的 harness 對 Qwen 直接寫	驗收	-2.0 個百分點	[-7.7, 0.0]	0／1／11	1.000
不用 harness 時，Qwen 對 gemma4:26b	單元	-0.9 個百分點	[-3.5, 0.0]	0／1／11	1.000
不用 harness 時，Qwen 對 gemma4:26b	驗收	-3.9 個百分點	[-9.1, 0.0]	0／2／10	0.500
gemma4:26b 用 harness 對不用 harness（含第一次研究）	單元	+10.3 個百分點	[3.5, 20.5]	7／0／5	0.016
gemma4:26b 用 harness 對不用 harness（含第一次研究）	驗收	+5.9 個百分點	[-16.0, 28.3]	5／2／5	0.453
改版：gemma4:26b 的 v4.2 對 v4／v4.1	單元	+8.6 個百分點	[-2.5, 24.4]	2／2／8	1.000
改版：gemma4:26b 的 v4.2 對 v4／v4.1	驗收	+9.8 個百分點	[-2.5, 27.7]	4／3／5	1.000
改版：Qwen 自己規劃的 v4.2 對 v4.1	單元	-12.1 個百分點	[-37.2, 8.5]	1／2／9	1.000
改版：Qwen 自己規劃的 v4.2 對 v4.1	驗收	+0.0 個百分點	[0.0, 0.0]	0／0／12	1.000

worker 的行為（表 13、表 14）：Qwen 當 worker 時工具錯誤率較高（R1＋Qwen 31.2%、Qwen 31.8%，gemma 23.3%），每個 job 的驗證失敗（7.2、5.3 對 3.5）、卡住被結束的 worker（1.17、0.75 對 0.36）、換新 worker 重試（1.71、1.17 對 0.75）都比較多；三者都沒有空白回覆（Ollama 的結構化輸出保證格式）。最明顯的差別是**空轉的修改**——內容通道送回跟檔案目前一模一樣的內容：Qwen 三次 63、52、60 次，幾乎全部是同一個 job 稍早用 code_write 建立的檔案（63/63、52/52、58/60），至少佔 11～16% 的模型呼叫；gemma 三次 18、16、12 次（3～5%）。Qwen 建立檔案時就把整個功能寫完，而 worker 的決策 session 看不到檔案內容、接著要求「實作某函式」，就變成空轉。遇到空轉之後，錯誤訊息會建議「先 code_view 確認，已經做好就回報完成」：gemma 的下一步 57% 先看檔案、17% 再改一次；Qwen 41% 先看檔案、19% 再改一次、22% 直接回報完成。主要的差別是空轉發生的次數（三次合計 Qwen 175、gemma 46），而不是遇到之後的反應。

表 13　worker 的行為（DevBench，每個 job 的平均）

條件	job 數	worker 步數	工具呼叫	工具錯誤率	驗證失敗	卡住被結束的 worker	空白回覆	沒改檔案就回報完成	換新 worker 重試
harness：gemma4:26b	36	41.1	32.2	23.3%	3.5	0.36	0.00	1.36	0.75
harness：DeepSeek-R1-32B 規劃＋Qwen2.5-Coder-32B worker	24	49.8	37.2	31.2%	7.2	1.17	0.00	0.79	1.71
harness：Qwen2.5-Coder-32B	12	41.3	30.4	31.8%	5.3	0.75	0.00	0.33	1.17
harness：gemma4:26b（v4.2）	12	47.9	38.6	20.3%	4.8	0.33	0.00	1.08	1.00
harness：Qwen2.5-Coder-32B（v4.2）	12	51.7	39.9	31.3%	7.2	0.83	0.00	0.50	1.58

時間（表 15）：每個 repo 平均 gemma 3.6 分鐘、Qwen 6.4 分鐘、R1＋Qwen 12.8 分鐘。規劃只佔一小部分（R1 每個 job 1.0 分鐘、Qwen 0.4、gemma 0.1）；R1 與 Qwen 交替時換模型平均每個 job 10 秒。R1 的計畫格式良好：需要程式修補的計畫 2/24（gemma 6/36、Qwen 1/12）。

表 15　時間與模型呼叫的分配（DevBench）

條件	job 數	規劃呼叫	規劃分鐘	worker 呼叫	worker 分鐘	載入模型秒數	計畫項數	計畫被程式修補
harness：gemma4:26b	36	2.0	0.1	57.9	2.8	0	5.1	6/36
harness：DeepSeek-R1-32B 規劃＋Qwen2.5-Coder-32B worker	24	1.9	1.0	69.6	11.4	10	6.5	2/24
harness：Qwen2.5-Coder-32B	12	2.0	0.4	60.4	5.9	0	6.6	1/12
harness：gemma4:26b（v4.2）	12	2.0	0.1	68.0	3.1	0	5.0	1/12
harness：Qwen2.5-Coder-32B（v4.2）	12	1.8	0.4	71.2	8.8	0	6.2	1/12

- **deepseek-r1:32b 關不掉思考**：think=false 時 5/5 仍然先思考（平均 618 字，跟 think=true 的 640 字差不多）。Ollama 的說明寫的是「think=false 時模型不會思考」[22]，示範的卻是 R1-0528 的 Qwen3 蒸餾版；這次用的 Qwen2.5 蒸餾版不吃這個開關。Ollama 仍然把思考放在獨立的欄位，回覆內容是乾淨的。
- **R1-32B 在 JSON 格式下會出現離題的答案**：「17 乘以 23」兩次都回答 "They are equal."，另外有殘留的格式（`\x08oxed{21}`）與奇怪的空白（"s s e n r a h"）——跟 R1 報告自承的「結構化輸出不如既有模型」[8] 一致。
- **gemma4:26b 的開關有效**：think=true 時每題思考 279～2,796 字；false 時不思考、0.2 秒。在這種簡單、要照格式回答的問題上，開 thinking 反而答錯兩題（5/5 → 3/5：把 harness 反轉錯、size_type 答成 size_t）。5 題太少，只能說「沒有看到幫助，慢了 3～30 倍」。
- **qwen2.5-coder:32b 沒有 thinking**：送 think=true 一律被 Ollama 拒絕（400「does not support thinking」）——所以設定畫面只在模型支援時顯示開關，送出前也會檢查。
- **主對話**：用 gemma4:e4b 實測，thinking 開著時主對話的回覆仍是合法的 JSON，思考內容（396 字）有被取出、顯示在右欄、沒有進入對話。

5.5 改版 v4.2（RQ6）

改了什麼。依據 5.1～5.3 的證據改了四件事（agent 程式的雜湊 6436be794225c705）：

1. **修改視窗以目標位置為中心**。產生 SEARCH/REPLACE 的 session 不再只看檔案的前 800 行，改成以下列三者之一為中心取 800 行：worker 最近一次 `code_view` 的範圍、SEARCH 對不上時「最像的段落」、或這一步提到的函式定義。
2. **既有專案的測試指令與範圍**：
   - `repo_map` 認得 django 與 sympy 自己的測試程式。
   - 建議指令裡的佔位符（`<測試模組>`）不能照抄成驗證指令。
   - 「這個 Python 沒有安裝 pytest」算環境問題，不派 worker 去修。
   - 測試檔 50 個以上的專案，整體驗證如果是「跑整套測試」，改成依序跑每一項自己的驗證指令。
3. **改動要小**。規劃與 worker 的規則加上「只改必要的地方」；重現問題的測試跟修正放在同一項。
4. **減少空轉**。寫入成功後，工具結果附上檔案現在有哪些類別與函式；內容通道送回一模一樣的內容時，明說可以直接回報完成。

量測方式。模型、設定與題目都不變：SWE-bench 24 題用 gemma4:26b 跑一次；DevBench 12 個 repo，gemma4:26b 與 Qwen 自己規劃各跑一次。結果在表 2～9 與表 10～15 的「(v4.2)」列。

SWE-bench：改到的機制都有效，解決數沒有變

表 17　SWE-bench 子集：gemma4:26b 的 v4.1／v4.1.1 與 v4.2

指標	v4.1／v4.1.1	v4.2
解決	7/24	7/24（只有 v4.2 解決 4 題、只有 v4.1 解決 4 題、兩次都解決 3 題；McNemar p=1.0）
每題時間（平均／中位數）	11.1／7.6 分鐘	3.9／3.4 分鐘
每題模型呼叫	86	61
SEARCH 對不上（其中目標在第 800 行之後）	125（44）	30（7）
工具錯誤率	37.4%	29.3%

指標	v4.1／v4.1.1	v4.2
每題卡住被結束的 worker	1.30	0.71
原始碼改的行數（中位數；參考解答 7）	18	12
修好了但弄壞既有測試	4 題	3 題
自己的整體驗證通過	4/23	16/24
自己驗證「通過」的題目，官方也解決	1/4	6/16
自己驗證「沒過」的題目，官方確實沒解決	14/19	7/8

- **修改視窗的修正直接解決了 5.1 的例子**。xarray-3677 這次 SEARCH 一次都沒有對不上，1.2 分鐘就解決（v4.1 對不上 12 次、只交出測試）；fmt-3901 從 30 次降到 3 次，也解決了。整體的 SEARCH 對不上少了 76%，目標在第 800 行之後的從 44 次降到 7 次。
- **解決數沒有變**。v4.2 新解決 4 題：django-12276、xarray-3677、fmt-3901、sphinx-8035。另外 4 題這次反而沒解：
  - flask-5014：第 2 個 SEARCH 區塊連續對不上，而且檔案裡找不到相似的段落；兩個 worker 都卡住之後跳過，交了白卷。
  - matplotlib-20488、fmt-2310：改錯地方。
  - django-13410：原始碼改了 74 行（v4.1 只改 14 行），沒有修好。

  兩次都解決的只有 3 題，跟「harness 對直接寫」的比較一樣（5.2 節）：單次執行的隨機性大於改版的效果，24 題、各跑一次量不出來。
- **時間少了將近三分之二**。主要原因是整體驗證通過的題目變多（16 對 4），不用再派修正的 worker；其次是 SEARCH 對不上與卡住的次數減少。
- **自己的驗證：「通過」變多，但仍然不可信；「沒過」變得可信**（表 7 的 v4.2 列）。v4.2 的驗證幾乎都改跑專案自己的測試程式，而且只跑相關的測試，所以 16 題「通過」。但其中只有 6 題官方真的解決，原因有兩個：
  - **大多數跑的是 agent 自己新寫的測試**，照自己的理解寫。例：astropy-8872 自己的測試 3 passed，官方的 F2P 也通過了，卻弄壞 4 個既有測試。
  - **3 題其實沒有跑到任何測試**：
    - django 的測試標籤寫成模組路徑（`django.forms.widgets`、`tests.test_sqlite`），`runtests.py` 回報「Ran 0 tests … OK」。
    - flask 的 `pytest -k` 篩不到任何測試，harness 改用靜態檢查，判定通過。
    - 其中 django-15561 帶著一個會讓 django 在建立測試資料庫時就當掉的修改「通過」了（它把 `deconstruct()` 回傳的 tuple 當成 dict 用）；官方計分一個測試都跑不起來，記成「patch 沒有成功套用」。

  反過來，自己驗證「沒過」的 8 題裡，有 7 題官方確實沒有解決。
- **還沒解決的問題**：
  - agent 會在專案根目錄留下自己的除錯或重現腳本（例如 sympy-16597 的 `debug_handler.py`、`diagnose_even.py`），v4.1 有 5/24 題、v4.2 有 3/23 題。計分不受影響，但實際使用時要清掉。
  - sympy-18199 的 patch 在每一個條件都帶著 1,666 個檔案權限的變更（644→755）。這是那個映像本身的特性，不影響套用與計分。

DevBench：沒有可信的變化（表 10、11、13～15 的 v4.2 列）

v4.2 的四個修正都是針對既有專案，DevBench 的情況不同：檔案都很短，不會有第 800 行之後的修改；整體驗證也大多只是一個測試指令。所以本來就不預期 DevBench 有大的變化。

- **gemma4:26b**：
  - 單元測試 23.3%（95% CI 5.6～45.6），驗收 33.3%。
  - 跟之前三次合併比：單元 +8.6 個百分點（95% CI −2.5～24.4；2 勝 2 負 8 平，p=1.0），驗收 +9.8（4 勝 3 負 5 平）。
  - 12 個 repo 中有 8 個的單元測試跟之前完全一樣。差別主要來自 graph-cpp：文件裡寫了兩種名稱，這次選對了（14/22），之前三次是 12、0、0。
- **Qwen 自己規劃**：
  - 單元測試 6.9%，驗收 11.8%。

  - 對 v4.1，單元 −12.1 個百分點（95% CI −37.2～8.5；1 勝 2 負 9 平），差別也主要來自 graph-cpp（這次 0/22，v4.1 16/22）。
  - 每個 repo 的時間在兩個方向都差很多。例：area_calculation 9.8 → 2.6 分鐘，people_management 9.0 → 29.2 分鐘。
- **Qwen 的空轉沒有減少**：57 次（v4.1 60 次），至少佔模型呼叫的 12.6%。v4.2 的提示改變的是「遇到空轉之後」的反應：

  | 遇到空轉之後的下一步（Qwen 自己規劃） | v4.1（60 次） | v4.2（57 次） | |---|---|---| | 先看檔案 | 42% | 40% | | 直接回報完成 | 15% | 39% | | 再改一次 | 25% | 11% | | 重複同樣的動作而被判卡住 | 10% | 4% |

  空轉發生的次數沒有變，因為根本原因在提示出現之前就已經發生：決策 session 看不到檔案內容，而 Qwen 建立檔案時就把功能寫完了。要解決，需要改協議本身（7.2 節）。gemma 也沒有減少：v4.2 一次 25 次、12 個 job，之前三次合計 46 次、36 個 job。

小結：v4.2 修的四個機制，在既有專案上都看得到作用——SEARCH 對不上、卡住、時間都明顯減少——但兩個基準的分數都沒有可信的變化。結果的變異很大：

- SWE-bench 上 v4.1 與 v4.2 各解決 7 題，只有 3 題相同；harness 對直接寫也是這樣。
- DevBench 的 graph-cpp 選對或選錯名稱，就差 14～16 個單元測試，佔全部 116 個的 12～14%。

在這種變異下，跑一次分辨不出改版的效果。每個條件至少要跑 3 次，或者題目要多很多（7.2 節）。

6. 討論

6.1 既有專案行不行？

使用者的假設是「DevBench 是從零寫，連 GPT-4 都做不太到；既有專案說不定比較好」。答案取決於「既有專案」是什麼樣子：

- **小而有測試的既有專案**（第一次研究的自建情境：幾個到十幾個檔案、測試完整、任務明確）：可行——harness 37/37，直接寫 71～82%。
- **大型真實專案的 issue**（SWE-bench：django 將近 5,000 個檔案、issue 常常只描述現象）：**還不行**——harness 解決 29～33%，跟直接寫（25%）沒有可信的差別。這個量級本身並不差：同樣沒有微調的 Qwen2.5-Coder-32B 配 OpenHands 在 Verified 上只有 7.0% [6]、配 SWE-agent 在 Multilingual 上 6.5% [5]（我們的子集較小、簡單題的比例照 Verified 分配，不能直接比）。但它表示 harness 在大型專案上的附加價值還沒有出來。

差別在哪裡？小專案裡 harness 的優勢是「每一項都跑測試、失敗就修」——前提是測試指令對、測試跑得快、跑的是相關的測試。大型專案裡這三個前提都常常不成立（5.2 節）：測試指令選錯、整套測試跑太久或有無關的失敗、agent 自己寫的測試照自己的錯誤理解寫。再加上大型檔案的修改視窗只有前 800 行（5.1 節問題二），harness 常常「找對了地方卻改不進去」。這些都是可以修的工程問題，不是方法本身的天花板。v4.2 修了其中幾項，機制都有效：SEARCH 對不上少了 76%，每題時間少了一半以上。但解決數沒有變（5.5 節），剩下的瓶頸是兩件事：

- 「改對」本身：改對檔案卻沒修好，仍然是最多的失敗類型（9 題）。
- 驗證的品質：agent 自己寫的測試照自己的理解寫；有的驗證其實沒有跑到任何測試。

在這些問題修好之前，**在大型專案裡用 harness 時，要由人指定測試指令與要跑的測試範圍，看驗證的輸出有沒有真的跑到測試，並且審查 patch**。

6.2 為什麼「coding 很強」的模型組合沒有比較好？

DevBench 上 R1＋Qwen 明顯比 gemma4:26b 差（單元 3.0% 對 14.7%，p=0.016）；SWE-bench 上兩者差不多（8/24 對 7/24），只有 C++ 的 4 題 R1＋Qwen 全部解決。原因有四層：

1. **協議的相容性比模型的榜單分數重要**。harness 的 worker 是兩段式：決策 session 以 JSON 決定動作，另一個內容通道寫出檔案。Qwen 建立檔案時習慣一次把整個功能寫完，而決策 session 看不到檔案內容、接著又要求「實作某函式」，內容通道就原樣送回，工具回報「沒有修改」。DevBench 上 Qwen 當 worker 的三次執行有 52～63 次這種空轉，幾乎全部發生在同一個 job 稍早建立的檔案上，至少佔 11～16% 的模型呼叫（gemma 3～5%）。這是 harness 的提示與協議照 gemma 的行為調出來的結果——**換模型需要重新調整協議，不是換上去就會變好**。v4.2 只在提示上處理

（寫入後列出檔案裡有哪些函式、空轉時明說可以回報完成），Qwen 遇到空轉後直接回報完成的比例從 15% 提高到 39%，但空轉的次數沒有變（60 → 57 次）：要改的是協議本身。反過來，SWE-bench 的修改多半是改既有檔案的幾行（不是從零建立檔案），空轉的機會少，Qwen 的程式能力就顯現出來（C++ 4/4）。

2. **R1 當規劃者沒有幫助**。同樣的 Qwen worker，R1 規劃 3.0%、Qwen 自己規劃 19.0%（p=0.25）。R1 的計畫格式良好（需要程式修補的 2/24），但計畫品質不是瓶頸——失分的主因是介面不符與 worker 的執行，規劃時多想一想解決不了。R1 的報告也寫明它在結構化輸出、工具使用與軟體工程任務上沒有優勢 [8]；我們用的 32B 蒸餾版沒有任何軟體工程的評測。
3. **量化與硬體**：四個模型都是 4-bit 量化；32B 的 dense 模型在這台機器（與 Isaac Sim 共用 GPU）約 42 tokens/秒，gemma4:26b（MoE，每個 token 只用約 4B 參數）快好幾倍。DevBench 每個 repo 的時間 R1＋Qwen 是 gemma 的 3.6 倍。
4. **榜單分數量的是別的事**。HumanEval、Aider 量的是「看得到完整程式、寫出或改對一段程式」；harness 的工作是「在工具迴圈裡逐步探索、照格式回報、照驗證結果修正」。SWE-smith 的作者也發現，未微調的 Qwen2.5-Coder-32B 在 agent 任務上很差，提升主要來自用 agent 軌跡微調 [5][6]。

6.3 Gemini 建議的逐項檢查

使用者與 Gemini 討論後的建議，逐項對照 harness 的現況與這兩次研究的數據：

Gemini 的說法或建議	harness 的現況	本研究的證據	判斷
「26B 直接寫比透過 harness 拆解更穩定，呼叫次數少很多」	—	DevBench：gemma4:26b 用 harness 單元 14.7%、直接寫 4.3%（7 勝 0 負，p=0.016）；自建情境 37/37 對 71～82%；SWE-bench 29% 對 25%（沒有差別）。呼叫次數：harness 每個 repo 約 60 次、直接寫約 5 次	「呼叫多很多」正確；「直接寫更穩定」與數據不符——不穩定的是長任務的分數本身（gemma 三次 9.5%～24.1%），直接寫也只有一次機會
Gemma 4 是通用多模態模型，格式漂移、頻繁重試	回覆由 Ollama 的結構化輸出（JSON schema 限制解碼）保證格式	三種模型的空白回覆都是 0；gemma 的工具錯誤率（DevBench 23%、SWE-bench 37%）比 Qwen（31%、46%）低	不成立：格式不是失敗原因；主要失分是介面不符與修改的正確性
R1 當規劃者最好（長鏈思考、自我質疑）	規劃者可以單獨換模型	同樣的 Qwen worker：R1 規劃 3.0%、Qwen 自己規劃 19.0%（p=0.25）；R1 的計畫格式良好（需要修補 2/24）但分數沒有比較好。R1 的報告自承結構化輸出、工具使用、軟體工程任務沒有優勢 [8]；32B 蒸餾版沒有軟體工程的評測	沒有證據支持；計畫品質不是這個 harness 的瓶頸
Qwen2.5-Coder 當 worker 最好（剛性的 JSON／工具格式、快）	worker 可以單獨換模型	DevBench：Qwen worker 的工具錯誤率、驗證失敗、卡住都比 gemma 多；空轉的修改 11～16% 的呼叫；每個 repo 的時間是 gemma4:26b 的 1.8 倍（Qwen 自己規劃）到 3.6 倍（R1 規劃）。SWE-bench：8/24 對 gemma 7/24，C++ 4/4	部分成立：程式能力確實強（C++ 的修改），但跟 harness 的寫檔協議合不來、在這台機器上慢；要先調協議（路線圖）
動態路由：簡單任務跳過規劃	沒有路由；簡單任務規劃只給 1～2 項	規劃只佔 job 時間的一小部分（gemma 每個 job 0.1 分鐘、R1 0.8～1.0 分鐘，worker 2.8～11.4 分鐘）	收益很小；而且跳過規劃就沒有「核准計畫」這個讓人審查的時機
規劃前要做唯讀探索	**已經有**：repo_map → 規劃者挑檔案讀 → 才規劃	SWE-bench：gemma 的 harness 改到正確檔案 18/24、看過 19/24，直接寫 12～14/24	已實作，而且是 harness 在大型專案上最明顯的優勢
思考與動作分離（只把結構化的計畫交給 worker）	**已經有**：Ollama 把 thinking 放在獨立欄位；回覆協議 thought／reply／action；worker 只拿到計畫的項目，看不到規劃者的思考	這次另外加了每個角色的 thinking 開關	已實作
用真實環境回饋（測試、linter、diff）而不是 reviewer 模型	**已經是核心設計**：每一項 code_check／code_test、失敗交還 worker、換 worker 重試，整體驗證失敗派修正、變差就還原	第一次研究：測試看得到時 74% 對 37%；這次：大型專案上測試指令選錯時，回饋反而是錯的（5.2 節）	方向正確、已實作；要補的是「測試指令與範圍選對」（v4.2 修了一部分）

Gemini 的說法或建議	harness 的現況	本研究的證據	判斷
失敗時交回規劃者重新規劃（Orchestrator re-planning）	失敗交回 worker、換新 worker，兩次都失敗就停下來問使用者（或跳過）	—	可以做，屬於「規劃的檢查器＋交回規劃者」的延伸（6.4 節）
邊讀邊產生漸進式小目標，不要一次拆完	一次拆完、使用者核准	—	跟「人核准計畫」的設計取捨；可以做「某一項失敗時重新規劃剩下的項目」
換成 R1＋Qwen-Coder-32B	—	DevBench 明顯變差（3.0% 對 14.7%，p=0.016）、每個 repo 的時間是 3.6 倍；SWE-bench 差不多（8/24 對 7/24）	不建議（在目前的 harness 上）；可以考慮「Qwen 當 C++ 修改的 worker」，但要先調協議

6.4 規劃階段的 self-refine

問題：worker 有測試可以檢查自己的結果、失敗再修；規劃階段沒有測試。能不能讓規劃者也「寫完計畫 → 檢查 → 修改」？

現在的規劃流程其實已經有一半：探索 → 規劃者挑檔案讀 → 產生計畫 → **程式檢查計畫**（`normalize_items`：路徑在不在、驗證指令能不能跑、要跑的測試檔存不存在、任務列出的檔案有沒有項目負責、有沒有新建任務沒列的原始檔）→ 有問題時**程式自己修補**（拿掉不能跑的驗證指令、補上漏掉的檔案…）→ 使用者核准。缺的是「把問題交回規劃者，讓它自己改一次」。

文獻的結論很一致：沒有外部回饋時，模型自己檢查、自己修改並不可靠，有時反而變差 [12][13]；自我修復的收益受限於模型給自己的回饋品質 [14]；規劃要靠外部的驗證器檢查、把錯誤回饋給模型 [15]。換句話說，**有用的是「用程式檢查計畫、把具體的問題交回去」，不是「請模型再想一次」**。推理模型（R1）的思考過程本身就包含大量的自我檢查，所以「請 R1 再檢查一次自己的計畫」跟它原本做的事重疊。

本研究的證據：

1. 計畫需要程式修補的頻率不高：DevBench gemma 6/36、R1 2/24、Qwen 1/12；SWE-bench 2/23 與 3/24。修補的內容多半是「驗證指令不能用」。這些都不是主要的失分原因——DevBench 的主要失分是**介面不符**，SWE-bench 的主要失分是**改不對、改不進去、驗證訊號錯誤**，現在的檢查都沒有涵蓋。
2. 規劃者換成會思考的 R1（同樣的 worker）沒有比較好（5.3 節）：「規劃時多想一想」在這個 harness 上沒有量到效果。
3. 規劃佔的時間很少（R1 每個 job 約 1 分鐘、worker 好幾分鐘）：多一次規劃呼叫的成本不高。

評估：

- 「請規劃者再看一遍自己的計畫」（純自我批評）：**不建議**——文獻顯示不可靠，R1 的思考已經在做，而且失分的主因不是規劃者「多想一次」就看得出來的。
- 「計畫的檢查器＋交回規劃者改一次」（LLM-Modulo 式）：**可行、便宜，建議做**，而且跟待辦清單的前兩項是同一件事：(1) 把 `normalize_items` 的檢查結果交回規劃 session 重寫一次（最多 1 次；沒有問題就不多花呼叫），剩下的才由程式修補；(2) 加上**介面覆蓋檢查**（待辦 1）：從規格抽出類別／函式／常數的名稱與參數，每一個都要出現在某一項的目標裡，沒有分配到的、拼法跟規格不一樣的，交回規劃者；(3) 加上**驗證範圍檢查**：既有專案的驗證指令要用專案自己的測試程式、只跑相關的測試（v4.2 已經在規則裡加了，檢查器可以強制）；(4) 既有專案：計畫提到要改的函式／類別要真的存在於指定的檔案（`symbol_index` 已經有），不存在就交回去；(5) **驗證指令要真的跑得到測試**：v4.2 的 16 題「通過」裡有 3 題其實一個測試都沒跑到（django 的測試標籤寫錯，`Ran 0 tests … OK`），規劃時先試跑一次每一項的驗證指令，跑到 0 個測試、或跑不起來的，就交回規劃者改。這正是「用程式檢查、把具體的問題交回去」能抓到、而「請模型再想一次」抓不到的錯誤。
- **時機**：照使用者的指示，等這次的測試結束後再做，而且當成獨立的改動來量：同一個模型、DevBench 重跑至少 3 次＋SWE-bench 子集，比較改動前後。

6.5 thinking 的設定建議

- **主對話**：關（預設）。主對話的回覆要照格式、要快；簡單的問答開 thinking 沒有看到幫助、慢很多。
- **Coding 規劃**：gemma4:26b 可以試開（規劃只佔 job 時間的一小部分），但這次沒有證據顯示有幫助（會思考的 R1 規劃沒有比較好）。

- **Coding worker**：關。worker 的呼叫次數多，開 thinking 會讓整個 job 慢很多。
- **deepseek-r1 系列**：開關沒有作用（總是會思考），當作「慢、但計畫格式穩定」的規劃模型看待。

6.6 效度威脅

- **樣本小、只跑一次**：SWE-bench 24 題、每個條件一次；DevBench 每個條件 1～3 次。單次執行的隨機性很大（DevBench gemma4:26b 三次的單元通過率 9.5%～24.1%；SWE-bench harness 與直接寫解決的題目幾乎不重疊）。所有比較都附信賴區間與配對檢定，沒有達到顯著的差別不應解讀為「一樣」或「不一樣」。
- **版本**：SWE-bench 的 gemma 結果由 v4.1（17 題）與 v4.1.1（7 題）組成；兩版只差一行正規表示式，那 17 題沒有任何會觸發的檔案，程式路徑相同。DevBench 第一次研究的 gemma 結果是 v4 在 f16 KV cache 上跑的，這次的是 v4.1 在 q8_0 上跑的。
- **環境的額外資訊**：harness 拿到一份「怎麼建置、怎麼跑測試」的說明（`.coding_agent.md`），標準的 SWE-bench 設定沒有；直接寫的對照組用不到它。
- **對照組的修正**：SWE-bench 的「不用 harness」對照組第一次執行的解析器太嚴格（同一個檔案的後續區塊沒寫路徑就算失敗），跑了 2 題後改成跟 harness 的 code_edit 一樣的比對方式，整組重跑。
- **時間上限**：每題 1 小時；慢的模型比較容易碰到（受 C++ 掃描器 bug 影響的題目之外，gemma 有 1 題跑滿，R1＋Qwen 沒有）。
- **計分的雜訊**：官方計分在高負載時可能被輸出交錯弄亂（5.1 節），已用固定規則檢查。
- **題目可能被模型看過**：SWE-bench 的題目與修正都在 GitHub 上，模型的訓練資料可能包含（資料污染）；直接寫的對照組也一樣受益。
- **改版的量測**：v4.2 是看過這 24 題的結果才改的，在同一批題目上變好不代表在沒看過的題目上也一樣（過度擬合）；要用新的題目確認（7.2 節）。

7. 結論與後續

7.1 結論

1. **既有專案（RQ1）**：
   - 小而有測試的既有專案可行（第一次研究）。
   - 大型真實專案的 issue：harness 解決 29～33%，同一個模型直接寫 25%，沒有可信的差別。
   - harness 的優勢是定位：改到正確檔案 18/24（直接寫 12～14/24），而且從不交白卷。
   - harness 的劣勢：改得太多，自己的驗證也不可信。
2. **模型組合（RQ2）**：R1 規劃＋Qwen2.5-Coder worker 不建議用在目前的 harness 上。
   - 從零寫明顯比 gemma4:26b 差（單元測試 3.0% 對 14.7%，p=0.016），既有專案差不多（8/24 對 7/24），每個 repo 慢 3.6 倍。
   - 原因是寫檔協議跟 Qwen 的習慣不合，不是 Qwen 的程式能力不足——它在 C++ 的 issue 上 4/4。
3. **推理模型當規劃者（RQ3）**：沒有幫助。計畫的品質不是這個 harness 的瓶頸。
4. **harness 對程式專用模型的價值（RQ4）**：DevBench 的單元測試，Qwen 用 harness +15.5 個百分點（3 勝 0 負，p=0.25，只跑一次），gemma4:26b +10.3 個百分點（7 勝 0 負，p=0.016）。
5. **thinking（RQ5 的前半）**：
   - deepseek-r1:32b 的開關沒有作用。
   - gemma4:26b 在簡單、要照格式回答的問題上開 thinking 沒有變準，慢 3～30 倍。
   - 建議預設關閉，規劃可以試開。
6. **規劃的 self-refine（RQ5 的後半）**：做「用程式檢查計畫，把具體的問題交回規劃者改一次」，不做純粹的自我批評。要檢查的內容：
   - 介面覆蓋。
   - 驗證指令是不是真的跑得到測試。
   - 既有專案裡，計畫提到的符號是否存在。

7. **改版 v4.2（RQ6）**：
   - 修到的機制都有效：SEARCH 對不上少 76%，每題時間的中位數 7.6 → 3.4 分鐘，卡住的 worker 少了將近一半。
   - 分數沒有變：SWE-bench 解決數不變（7/24），DevBench 沒有可信的變化。
   - 暴露出新的問題：驗證沒有跑到任何測試，卻判定通過。
8. **方法上的教訓**：
   - 這個規模（24 題、12 個 repo、每個條件 1～3 次）只量得出很大的差別，例如 R1＋Qwen 對 gemma 在 DevBench 的 −11.6 個百分點。
   - 過程指標（SEARCH 對不上、空轉、卡住、時間、自己的驗證跟官方結果是否一致）對改版的反應，比最後的分數靈敏得多，適合當改版的第一道檢查。
   - 分數有沒有改善，要用重複的量測與沒看過的題目確認。

7.2 路線圖與後續評測

依「直接解決這次找到的問題 × 實作成本」排序；完整的表（含量測目標）在《評估報告》第八、九節。

1. **驗證要真的跑到測試**：
   - 驗證輸出顯示 0 個測試（`Ran 0 tests`、`collected 0 items`、`-k` 篩不到）時，不能算通過。
   - django 的測試標籤要對照 `tests/` 底下的目錄名稱。
   - 規劃時先試跑一次每一項的驗證指令。
2. **既有專案的回歸檢查**：
   - 修改前後各跑一次被改到的模組原本的測試，通過的變少就交回修正或還原。
   - agent 寫的重現測試，要先確認在修正前會失敗。
3. **計畫的檢查器＋交回規劃者改一次**（self-refine 的做法），包含第 1 項的檢查與介面覆蓋（待辦 1）。
4. **讓協議適用於不同模型**：
   - worker 建立檔案後，決策 session 看得到檔案的大綱。
   - 這一步要「實作」的函式已經存在、而且不是空殼時，不呼叫內容通道，直接告訴 worker。
5. **清掉 agent 留下的檔案**：報告列出計畫以外新建的檔案（除錯、重現腳本），由使用者決定要不要刪。
6. **使用者要求先記錄、還沒有開始的 4 項**（`doc/待辦清單.md`）：
   - 介面清單與一致性檢查。
   - 長任務 test-first。
   - agent 執行的程式加上網路隔離。
   - 用 amr_base 的真實任務驗證。

後續評測：

- 每個條件至少跑 3 次。
- 用沒看過的題目確認 v4.2 以後的改版，避免過度擬合：Verified 擴大到 50 題，Multilingual 的 C／C++ 擴大到 15～20 題。
- 最重要的是 amr_base 的 10～20 個真實任務。
- 再試別的模型之前，先做第 4 項，否則量到的是「協議合不合」，不是模型的能力。

參考文獻

編號與 `references/README.md` 相同；PDF 與網頁的副本在 `references/`。

[1] Li et al. *Prompting Large Language Models to Tackle the Full Software Development Lifecycle: A Case Study*（DevBench／DevEval）. arXiv:2403.08604, 2024. [2] ByteDance Seed 等. *NL2Repo-Bench: Towards Long-Horizon Repository Generation Evaluation of Coding Agents*. arXiv:2512.12730, 2025. [3] Jimenez et al. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR 2024. arXiv:2310.06770. [4] OpenAI. *Introducing SWE-bench Verified*. 2024. https://openai.com/index/introducing-swe-bench-verified/ [5] Yang et al. *SWE-smith: Scaling Data for Software Engineering Agents*（SWE-bench Multilingual 的出處）. arXiv:2504.21798, 2025. [6] Pan

et al. *Training Software Engineering Agents and Verifiers with SWE-Gym*. arXiv:2412.21139, 2024. [7] Hui et al. *Qwen2.5-Coder Technical Report*. arXiv:2409.12186, 2024. [8] DeepSeek-AI. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. arXiv:2501.12948v2, 2026. [9] Madaan et al. *Self-Refine: Iterative Refinement with Self-Feedback*. NeurIPS 2023. [10] Shinn et al. *Reflexion: Language Agents with Verbal Reinforcement Learning*. NeurIPS 2023. [11] Chen et al. *Teaching Large Language Models to Self-Debug*. ICLR 2024. [12] Huang et al. *Large Language Models Cannot Self-Correct Reasoning Yet*. ICLR 2024. [13] Kamoi et al. *When Can LLMs Actually Correct Their Own Mistakes? A Critical Survey of Self-Correction of LLMs*. TACL 2024. [14] Olausson et al. *Is Self-Repair a Silver Bullet for Code Generation?* ICLR 2024. [15] Kambhampati et al. *LLMs Can't Plan, But Can Help Planning in LLM-Modulo Frameworks*. ICML 2024. [16] Bairi et al. *CodePlan: Repository-level Coding using LLMs and Planning*. FSE 2024. [17] Jiang et al. *Self-planning Code Generation with Large Language Models*. arXiv:2303.06689, 2023. [18] Wang et al. *Planning In Natural Language Improves LLM Search For Code Generation*. arXiv:2409.03733, 2024. [19] Xia et al. *Agentless: Demystifying LLM-based Software Engineering Agents*. arXiv:2407.01489, 2024. [20] Yang et al. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. NeurIPS 2024. [21] Wang et al. *OpenHands: An Open Platform for AI Software Developers as Generalist Agents*. ICLR 2025. [22] Ollama. *Thinking*. 2025-05-30. https://ollama.com/blog/thinking [23] Ollama 模型頁：deepseek-r1、qwen2.5-coder. https://ollama.com/library/deepseek-r1、https://ollama.com/library/qwen2.5-coder [24] Hobbhahn. SWE-bench Verified mini. https://huggingface.co/datasets/MariusHobbhahn/swe-bench-verified-mini [25] Epoch AI. *How to run SWE-bench Verified in one hour on one machine*. 2025. https://epoch.ai/blog/swebench-docker

附錄 A：SWE-bench 子集的題目

見 `datasets/swebench_subset/instances.md`（題目、語言、專案、難度、評分測試的數量）與 `README.md`（抽樣規則與評測協定）。

附錄 B：每一題、每個 repo 的結果

表 B1　SWE-bench 子集：每一題的結果

題目	語言	難度	harness：gemma4:26b	harness：DeepSeek-R1-32B 規劃＋Qwen2.5-Coder-32B worker	不用 harness：gemma4:26b	不用 harness：Qwen2.5-Coder-32B	harness：gemma4:26b（v4.1，C++ 掃描器 bug）	harness：gemma4:26b（v4.2）
astropy__astropy-8872	Python	15 min - 1 hour	○	✅	✅	✅	○	○
django__django-11141	Python	15 min - 1 hour	○	⊘	○	○	—	○
django__django-11734	Python	15 min - 1 hour	✗	✗	○	○	—	○
django__django-12276	Python	<15 min fix	○	✅	⊘	○	—	✅

題目	語言	難度	harness：gemma4:26b	harness：DeepSeek-R1-32B 規劃＋Qwen2.5-Coder-32B worker	不用 harness：gemma4:26b	不用 harness：Qwen2.5-Coder-32B	harness：gemma4:26b（v4.1，C++ 掃描器 bug）	harness：gemma4:26b（v4.2）
django__django-13410	Python	<15 min fix	✅	✅	✅	✅	—	○
django__django-14089	Python	<15 min fix	✅	✗	⊘	✗	—	✅
django__django-15561	Python	15 min - 1 hour	✗	✗	✗	✅	—	✗
matplotlib__matplotlib-20488	Python	15 min - 1 hour	✅	✗	✗	✗	—	✗
pallets__flask-5014	Python	<15 min fix	✅	✅	⊘	⊘	—	⊘
pydata__xarray-3677	Python	15 min - 1 hour	✗	✗	✅	✅	—	✅
pytest-dev__pytest-10051	Python	15 min - 1 hour	○	○	○	○	—	○
scikit-learn__scikit-learn-14496	Python	<15 min fix	✅	⊘	✅	⊘	—	✅
sphinx-doc__sphinx-10435	Python	<15 min fix	○	✗	✗	✗	—	○
sphinx-doc__sphinx-8035	Python	15 min - 1 hour	○	○	○	○	—	✅
sympy__sympy-16597	Python	1-4 hours	✗	○	✗	✗	—	○
sympy__sympy-18199	Python	1-4 hours	○	○	○	○	—	○
jqlang__jq-2235	C	—	○	○	○	○	—	○
micropython__micropython-13039	C	—	○	○	⊘	✅	—	○
redis__redis-12472	C	—	○	○	○	○	⊘	○
redis__redis-13115	C	—	✗	✗	⊘	⊘	⊘	✗
fmtlib__fmt-1683	C++	—	○	✅	✅	✅	○	○
fmtlib__fmt-2310	C++	—	✅	✅	✗	○	—	✗

題目	語言	難度	harness：gemma4:26b	harness：DeepSeek-R1-32B 規劃＋Qwen2.5-Coder-32B worker	不用 harness：gemma4:26b	不用 harness：Qwen2.5-Coder-32B	harness：gemma4:26b（v4.1，C++ 掃描器 bug）	harness：gemma4:26b（v4.2）
fmtlib__fmt-3901	C++	—	○	✅	✅	○	—	✅
nlohmann__json-4237	C++	—	✅	✅	⊘	⊘	—	✅

✅ 解決；○ 改到該改的檔案但沒有通過評分測試；✗ 改了別的檔案；⊘ 沒有改任何東西

表 B3　DevBench：每一次執行

附錄 C：重現

見 `README.md` 的「重跑實驗」與「重新算出全部的表與圖」。原始紀錄在 `results/raw/`（每個結果目錄有 `HARNESS_VERSION.txt` 記錄用哪一份凍結副本跑的；SWE-bench 的目錄另有 `patches/` 與 `official_eval/`——每一題的官方計分紀錄）。

附錄 D：版本與雜湊

版本	agent 程式的雜湊	內容	用在
v4.1	5b37b79521737866	第一次研究的最後版本	DevBench（E8）全部；SWE-bench gemma 沒有觸發 bug 的 17 題、兩個對照組（不用 harness，與版本無關）
v4.1.1	ed861606750f20b1	v4.1＋C++ 掃描器的一行修正	SWE-bench：gemma 受影響的 7 題、R1＋Qwen 全部 24 題
v4.2	6436be794225c705	v4.1.1＋既有專案的四個修正＋每個角色的模型與 thinking 開關	E10（5.5 節）